

Part B: end-of-session check-in

Build spec for Part B, extending the existing Part A stack.

Scope

- Extend Part A's stack. No parallel tables or second app.
- Part A's four Google API traps all apply. Part B has its own template and one-time Verified step.
- Part B measures what landed: a second form, released at the end.

Form fields

Fixed order: click-first surveys complete at 89% vs 83%; early sensitive items raise abandonment. Don't reorder.

1. Email + timestamp (auto).
2. Confidence, "I could explain today's topic to someone": 7-point scale.
3. One-sentence takeaway: short text.
4. Rotating slot: changes weekly.
5. Peer shoutout, "who helped you today?": optional text.
6. "I'd like someone to check in with me": checkbox, last. - 7 points: 5-point scales cause interpolation, and this gets graphed. - Rotation fights fatigue (3 to 4 questions cuts completion 18%); nobody sees over four questions. - The teacher question runs most often (Deliverable 2) because only someone present can answer it.

Design invariants

Part A's invariants hold. These add ethical limits.

1. **Asking for help never lowers any participation signal.** Exclude the checkbox from every count, score, rate and aggregate, in code.
2. **No named recipient in `config/help_routing.json`, no help checkbox.** Omit it and log why. The Director of Programs can serve today (no Fellow-support role exists, per CU on 2026-08-10).
3. No AI judges an individual's free text. Gemini only clusters muddiest-point answers.
4. Free text is counted, never graded (grading penalizes ESL and neurodivergent fellows).
5. Shoutouts are third-party data: same protection, never shown to the named person without a data-owner decision.
6. The help checkbox never reaches AI, a general report or a CSV export.

Deliverable 1: Form structure and provisioning

- `form_template` and `session_form` gain part (a|b): separate templates and Verified confirmations, one form per part per session.
- Build via `forms.batchUpdate` → `createItem`, by `location.index`:
 - 0 confidence: `scaleQuestion` `low`: 1, `high`: 7, `lowLabel`: "Not at all", `highLabel`: "Easily", `required`: true

- 1 takeaway: `textQuestion`, `paragraph`: false, `required`: true
- 2 rotating: same, titled with that week's text
- 3 shoutout: `textQuestion`, `paragraph`: false, `required`: false
- 4 help: `choiceQuestion`, `type`: "CHECKBOX", one option, `required`: false

Question ID mapping (new trap)

- Responses are keyed by `questionId`. Nobody has verified whether `files.copy` keeps IDs; don't hardcode them.
- After provisioning, `forms.get` and store `questionId` → slot (`confidence`, `takeaway`, `rotating`, `shoutout`, `help`) in `form_question_map`, matched by item index, not title.
- **Missing or incomplete map: refuse to ingest that form.**
- Snapshot the exact shown text into `question_text`; never rebuild it from config.

Deliverable 2: Rotation engine

`config/rotation.json`, owned by the Director of Programs:

```
{
  "version": "1.0.0",
  "owner": "adiah@civicsunplugged.org",
  "status": "DRAFT – NOT APPROVED",
  "schedule": {
    "teacher_question": [1, 4, 7, 10],
    "muddiest_point": [2, 5, 8],
    "application": [3, 6, 9]
  },
  "fixed_text": {
    "muddiest_point": "What's still unclear?",
    "application": "One way you'd use this in your project"
  },
  "wrap": true
}
```

- Weeks past the schedule wrap; document it.
- Week comes from `session.week_index`, never calendar dates.
- Teacher-question week with empty `session.teacher_question`: provisioning refuses, naming the session. No generic fallback.
- Startup validation: each week 1..N exactly once.

Deliverable 3: Schema additions

Migrations only; nothing destructive.

- `form_template`: add `part`. `session_form`: add `part`, unique (`session_id`, `part`).
- `session`: add `week_index` int, `teacher_question` text null.
- `form_question_map`: id uuid PK, `form_id`, `question_id`, `slot`, `rotating_kind` (`teacher_question|muddiest_point|application`, null), `question_text`, `item_index` int, `recorded_at`. Unique (`form_id`, `question_id`) and (`form_id`, `slot`).

- `checkin_b` (immutable): `checkin_b_id` uuid PK, `source_event_id` UNIQUE, `source` (`forms_api|csv`), `submitted_email`, `submitted_at_utc`, `session_id` FK null, `session_match` (`matched|none|ambiguous`), `confidence_raw` int 1 to 7 null, nullable text `takeaway_text`, `rotating_kind`, `rotating_text`, `shoutout_text`, `latency_seconds` null, `extra_fields` jsonb, `load_id` FK, `ingested_at`. No help column.
- `peer_shoutout`: `shoutout_id` uuid PK, `checkin_b_id` FK, `raw_text`, `named_fellow_id` FK null, `match_method` (`exact_name|manual|unresolved`), `confidence`, `resolved_by`, `resolved_at`. One row per name.
- `muddiest_theme`, `muddiest_theme_member`: per session label, summary, member `checkin_b_ids`, model, prompt version, `generated_at`; supersede on regenerate.
- RLS: `checkin_b` and `peer_shoutout` get Part A's `TODO(access)` stub.

Deliverable 4: Ingestion

- `cufa pull --part b`: Part A's watermark, pagination and `source_event_id` hashing.
- Confidence: int 1 to 7. Out of range: NULL, raw into `extra_fields`, warn. Never clamp.
- Takeaway, rotating: verbatim, plus mapped `rotating_kind`.
- Shoutouts: Deliverable 6. Help: Deliverable 5.
- Session match, identity, latency: as in A.
- A and B are independent: either alone is valid, B never backfills A. Join on (`fellow`, `session`) at read time.

Deliverable 5: Help requests

`help_request`: `help_request_id` uuid PK, `fellow_id` FK null, `submitted_email`, `session_id` FK null, `submitted_at_utc`, `status` (`open|acknowledged|closed`), nullable `acknowledged_by`, `acknowledged_at`, `note`.

- Email the recipient the moment it's ingested.
- Console shows why when the checkbox is omitted.
- Stricter RLS than any table; not covered by the "visible to full-time staff" default.
- Never touched by AI, in `cufa report`, a CSV export or an aggregate, or read by participation code.
- Access-gated console screen to acknowledge and close.
- Notifications carry fellow name and session only.
- Leave `TODO(retention)`; invent no period.

Deliverable 6: Peer shoutout resolution

- Split on commas, `and`, `&`, newlines; keep each fragment.
- Normalize (trim, collapse whitespace, casefold); match full then first names, cohort only.
- Unambiguous match: `exact_name`, confidence 1.0.
- Ambiguous (two Jordans, goes to review) or no match (guests): `unresolved`, not an error.
- Console linking writes `manual` with the resolver's identity.
- No ranking, points or display. The ADR notes future recognition should rank by giving, not receiving.

Deliverable 7: Confidence trend

- Store `confidence_raw` as int 1 to 7; no rescaling.
- Read-time view: raw per fellow per session; median and IQR per cohort per week (ordinal, no mean).
- `cufa report --confidence` prints the weekly trend.
- Docs and console say trends and dips (6 → 3) matter, not single scores.
- Straight-lining (same value 4+ sessions running): data-quality flag, review screen only, never a signal.

Deliverable 8: Muddiest-point clustering

- Input: one session's `rotating_text` where `rotating_kind = 'muddiest_point'`, text only (no names, emails, `fellow_ids`, per-person counts).
- Output: 2 to 5 themes (label, one-sentence summary, answer indices), structured JSON.
- Part A's Gemini setup (`gemini-2.5-flash`, `temperature=0`, versioned prompt per row, 429 backoff). No key: no themes, clear message.
- Teacher view only (`cufa themes`, session detail); fellow view is the data owner's call.

Deliverable 9: Console additions

- Session form: `week_index`; `teacher_question` shown when needed, warning if empty.
- Session detail: Part B beside Part A, each with status, URL, QR code, publish-verified indicator.
- Rotation preview, response view (confidence, takeaways, themes), shoutout linking.
- Help requests: separate, access-gated, visually distinct.
- Where staff might add fields, say six is by design, citing completion numbers.

Deliverable 10: CLI

```
cufa provision --session <id> --part b [--dry-run]
cufa pull      --session <id> --part b
cufa themes    --session <id> [--regenerate]
cufa shoutouts review | link --shoutout <id> --fellow <id> --by <email>
cufa help-requests list | ack --id <id> --by <email> --note "<text>"
cufa report    --cohort <id> [--confidence] [--json]
```

Deliverable 11: Test harness

- `FakeGoogleClient` adds Part B question types and an ID scheme: preserved or regenerated by `files.copy`.
- Fixtures add: confidence 1 to 7 plus 0, 8, "four", blank; takeaways (substantive, one-word, whitespace, emoji, very long); all rotating kinds over ten weeks; shoutouts (single, comma list, "X and Y", non-roster, ambiguous, blank); a few help checks; a 5-session straight-liner; A-only and B-only fellows; one incomplete map.

Deliverable 12: Tests

Part A's tests still pass. Offline, with fakes.

- Mapping: both ID schemes; incomplete map refuses; `question_text` survives config change.
- Rotation: weeks 1 to 10 right; reschedule keeps question; empty teacher question blocks; bad config rejected.
- Fields: out-of-range → NULL plus raw; blank shoutout adds no row; multi-name splits; ambiguous and non-roster → `unresolved`; straight-lining flagged at 4, outside signals.
- Help: omitted without routing; notifies on ingest; notice lacks takeaway, confidence, shoutout; absent from exports; no participation query reads it (inspect query paths); never sent to AI.
- AI: no email, name or `fellow_id` in payload; no key completes; regeneration supersedes.
- Independence: A-only and B-only ingest, no backfill.

Deliverable 13: Documentation

- `docs/setup/part-b-form.md`: template, Verified step, rotation, teacher's weekly prep.
- `docs/safeguarding.md`, for CU staff: who's notified, what they get, access, no-recipient-no-field, open retention.
- `docs/decisions.md` ADRs: field-order-from-completion-research, 7-point-over-5-point, rotation-over-every-question-every-week, question-ID-map, help-requires-named-recipient, help-excluded-from-all-signals, AI-clusters-content-not-people, shoutouts-collected-not-ranked.
- Add the question-ID trap to `README.md` and `docs/google-api-traps.md`.

Constraints

- Part A's constraints hold. **Never log help request content, even at DEBUG.**

Acceptance criteria

1. `make setup` & `make demo` runs A and B end to end with no Google account or `GEMINI_API_KEY`.
2. Rerun adds no rows to `checkin_b`, `peer_shoutout` or `help_request`.
3. Every fixture Part B response lands in `checkin_b`.
4. Provisioning fails loudly on a teacher-question week without a question.
5. No recipient, no help checkbox.
6. Mapping works under both ID schemes.
7. Tests prove `help_request` is in no export, report or participation computation.
8. Tests prove the clustering payload has no identifying field.
9. `make test` passes offline.
10. A non-engineer can read `docs/safeguarding.md`.

Out of scope

- Gamification, leaderboards, points, streaks, public shoutouts.
- Cross-source participation scoring (Director of Programs owns weighting); at-risk labels.
- Fellow-facing views; Slack or Zoom, auto-posting, reminders.
- Dashboards beyond console and report; Supabase Cloud or public hosting.

See also

- `docs/implementation/part-a-passphrase-checkin.md`